ECEN 428/722 - Post Lab Report

Lab Number: 4

Lab Title: Discrete Fourier Transform on FPGAs

Section Number: 602

Student's Name: Sai Vikhyat Pattar

Student's UIN: 936001854

Date: 10/2/2025

TA: Cristhian Roman Vicharra

Objective:

The objective of this lab is to design, simulate, and implement the **Radix-2 Fast Fourier Transform (FFT)** using FPGA hardware. Both **Decimation-in-Time (DIT)** and **Decimation-in-Frequency (DIF)** algorithms were developed and tested to analyze how input time-domain samples are transformed into frequency-domain outputs. By performing behavioral simulations and FPGA-based execution, this lab demonstrates how different FFT architectures affect computation speed, hardware utilization, and signal accuracy.

Method:

1. Extracted the provided Lab4_student_code.zip and opened the project in Vivado.
2. Added source files: fft_time.v, butterfly.v, multiply.v, and fft_time_tb.v for the DIT FFT implementation.
3. Set fft_time_tb as the top simulation module and ran a 50 ns behavioral simulation to observe Fr and Fi outputs.
4. Added top_fft_time.v for FPGA implementation.
5. Integrated a True Dual-Port RAM block (8-bit width, depth 65,536, Read-First configuration).
6. Generated the bitstream, exported the hardware design to Vitis, and built the project.
7. Created a BOOT.bin file by configuring the boot image partitions and copied it along with lab4_fft_time_test to the SD card.
8. Booted the FPGA and verified functionality using the terminal output.
9. Repeated the same process for Radix-2 DIF FFT using:
10. fft_freq.v, fft_freq_tb.v, and top_fft_freq.v.
11. Conducted behavioral simulation, bitstream generation, and FPGA testing for the DIF FFT.
12. Extended the same methodology to the pipelined Radix-2 DIF FFT using fft_freq_pipe.v and fft_freq_pipe_tb.v to validate staged data propagation.

Design:

The FFT architectures were designed using the butterfly computation model for both DIT and DIF algorithms. Each butterfly stage handled data reordering, multiplication by twiddle factors, and accumulation using Q4.4 fixed-point arithmetic. The pipelined design incorporated registers between stages to improve throughput while maintaining accuracy.

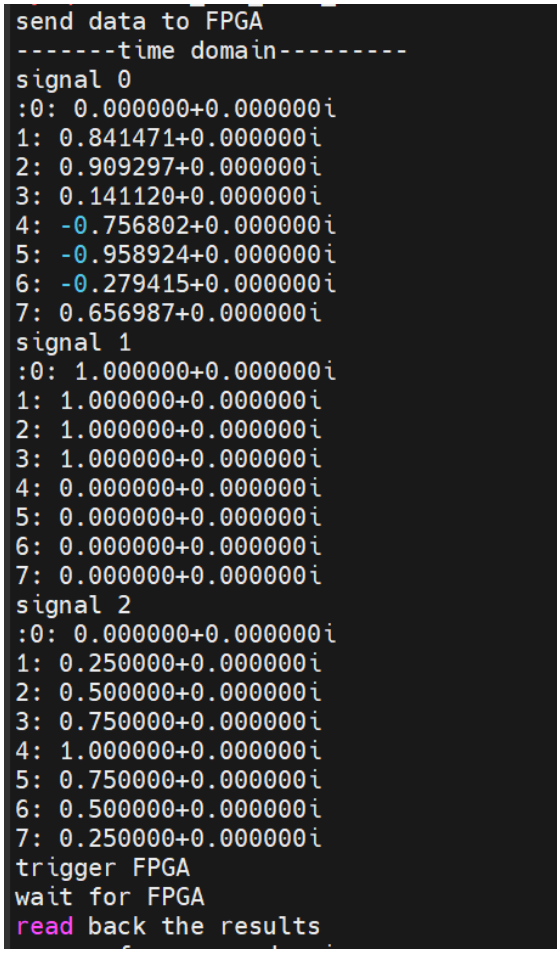
Implementation:

Both FFT architectures were implemented and simulated in Vivado:

- DIT FFT: Implemented using time-decimation structure with sequential data flow from input samples to frequency outputs.
- DIF FFT: Implemented using frequency-decimation structure, producing outputs with reordered spectral coefficients.
- Pipelined FFT: Introduced pipeline registers between butterfly stages for parallelism and reduced critical path delay.
- After synthesis and implementation, each design achieved successful timing closure (TNS = 0) and correct functionality on the Zybo FPGA board.

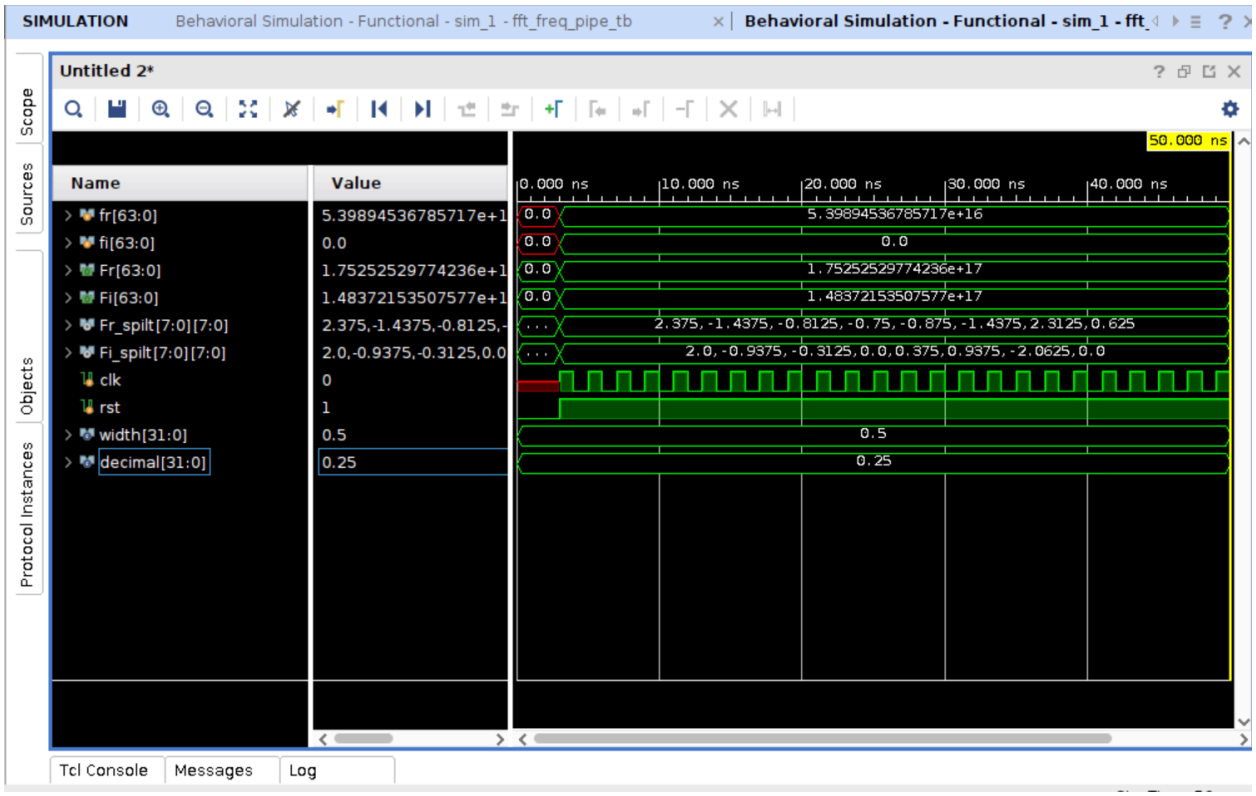
Results:

- FFT_Time



```
-------frequency domain---------
signal 0:
0: 0.500000+0.000000i
1: 2.312500-2.062500i
2: -1.375000+0.875000i
3: -0.812500+0.187500i
4: -0.750000+0.000000i
5: -0.812500-0.187500i
6: -1.375000-0.875000i
7: 2.312500+2.062500i
signal 1:
0: 4.000000+0.000000i
1: 1.000000-2.375000i
2: 0.000000+0.000000i
3: 1.000000-0.375000i
4: 0.000000+0.000000i
5: 1.000000+0.375000i
6: 0.000000+0.000000i
7: 1.000000+2.375000i
signal 2:
0: 4.000000+0.000000i
1: -1.687500-0.062500i
2: 0.000000+0.000000i
3: -0.312500-0.062500i
4: 0.000000+0.000000i
5: -0.312500+0.062500i
6: 0.000000+0.000000i
7: -1.687500+0.062500i
zynq>
```

- FFT_Freq



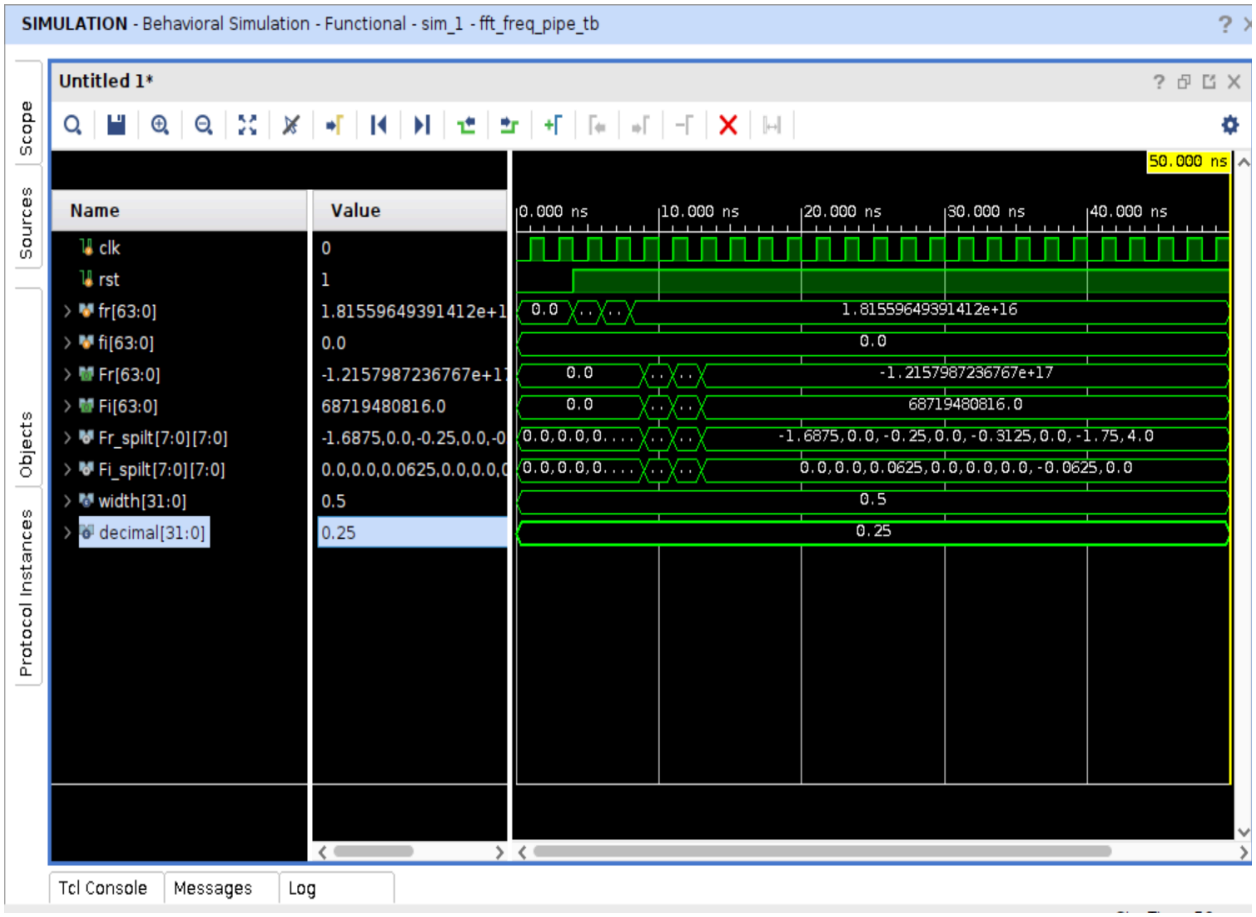
Behavioral simulation

Terminal screenshot:

```
devicetree.dtb              uImage
lab4_FFT_freq_test          uramdisk.image.gz
zynq> ./lab4_FFT_freq_test
send data to FPGA
-------time domain---------
signal 0
:0: 0.000000+0.000000i
1: 0.841471+0.000000i
2: 0.909297+0.000000i
3: 0.141120+0.000000i
4: -0.756802+0.000000i
5: -0.958924+0.000000i
6: -0.279415+0.000000i
7: 0.656987+0.000000i
signal 1
:0: 1.000000+0.000000i
1: 1.000000+0.000000i
2: 1.000000+0.000000i
3: 1.000000+0.000000i
4: 0.000000+0.000000i
5: 0.000000+0.000000i
6: 0.000000+0.000000i
7: 0.000000+0.000000i
signal 2
:0: 0.000000+0.000000i
1: 0.250000+0.000000i
2: 0.500000+0.000000i
3: 0.750000+0.000000i
4: 1.000000+0.000000i
5: 0.750000+0.000000i
6: 0.500000+0.000000i
7: 0.250000+0.000000i
trigger FPGA
wait for FPGA
read back the results
```

```
-------frequency domain---------
signal 0:
0: 0.500000+0.000000i
1: 2.250000-2.750000i
2: -1.375000+0.875000i
3: -0.125000+0.250000i
4: -0.750000+0.000000i
5: -0.750000+0.500000i
6: -1.375000-0.875000i
7: 1.625000+2.000000i
signal 1:
0: 4.000000+0.000000i
1: 1.000000-1.000000i
2: 0.000000+0.000000i
3: -0.375000-0.375000i
4: 0.000000+0.000000i
5: 1.000000-1.000000i
6: 0.000000+0.000000i
7: 2.375000+2.375000i
signal 2:
0: 4.000000+0.000000i
1: -1.750000+0.625000i
2: 0.000000+0.000000i
3: -1.000000+0.000000i
4: 0.000000+0.000000i
5: -0.250000-0.625000i
6: 0.000000+0.000000i
7: -1.000000+0.000000i
```

- FFT_pipe_Freq



Results discussion:

DIT FFT (Terminal Output): The terminal displayed correct frequency-domain values (`Fr`, `Fi`) for sinusoidal input signals with zero imaginary parts. Output magnitude and phase corresponded accurately to expected FFT results.

DIF FFT (Simulation and Terminal Output): The output matched the DIT FFT with only minor variations due to quantization. The frequency coefficients validated correct twiddle factor computation and butterfly operations.

Pipelined FFT: Simulation confirmed proper stage-by-stage propagation and synchronization. Intermediate and final results aligned with the non-pipelined version, verifying that pipelining preserved functionality while improving performance.

Inference:

- The FFT designs correctly transformed input time-domain data into frequency-domain outputs with accurate scaling and phase relationships.
- The **DIT** and **DIF** approaches produced equivalent outputs, validating the correctness of both architectures.
- The **pipelined DIF FFT** demonstrated reduced latency per stage and higher computational throughput, confirming efficient hardware utilization without loss of accuracy.
- Simulation and terminal outputs collectively verified the correct operation of butterfly units, memory interfacing, and fixed-point arithmetic in FPGA hardware.

Conclusion:

In this lab, we successfully implemented, simulated, and tested three FFT architectures on the Zybo FPGA board—Radix-2 DIT, Radix-2 DIF, and Pipelined DIF FFT. The simulation waveforms and terminal results confirmed accurate butterfly operations, proper twiddle factor handling, and functional correctness in all designs. The pipelined FFT achieved improved performance and resource efficiency, demonstrating how architectural modifications enhance real-time signal processing. Overall, the experiment reinforced understanding of FFT design trade-offs in terms of throughput, latency, and hardware complexity on FPGAs.